

[CS230] Sequence Models: Attention Mechanism

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1-10.10 RNN, LSTM, Word2Vec, BLEU - *Completed*
- **10.11 Attention Mechanism**
 - * The Bottleneck Problem in Seq2Seq
 - * Dynamic Context Vector ($c^{(t)}$)
 - * Alignment Scores & Attention Weights (α)
 - * Implementation: Bahdanau Attention

Chapter 11. Transformer Network - *Next Part*

지난 시간 복습 및 연결

지난 시간에 배운 Seq2Seq 모델은 혁신적이었지만, 문장이 길어지면 성능이 급격히 떨어지는 ‘**병목(Bottleneck)**’ 현상을 겪었습니다. 인코더가 100 단어짜리 긴 문장을 고정된 크기의 벡터 하나에 억지로 압축해야 했기 때문입니다.

생각해 봅시다. 여러분이 긴 문장을 번역할 때, 문장 전체를 한 번 읽고 외운 다음 눈을 감고 번역합니까? 아닙니다. ”지금 번역하려는 단어와 관련된 원문 부분을 그때그때 다시 쳐다보면서 (**Attend**)” 번역합니다. 이 과정을 구현한 것이 바로 어텐션(**Attention**)입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 긴 시퀀스 정보 손실을 해결하고 성능을 극대화하는 어텐션 메커니즘을 다룹니다.

- **동적 문맥:** 매 타임 스텝마다 다르게 계산되는 문맥 벡터 $c^{(t)}$ 의 개념을 이해합니다.
- **가중치:** 인코더의 특정 단어에 얼마나 집중할지 나타내는 어텐션 가중치(α)를 정의합니다.
- **구조:** 인코더의 모든 은닉 상태를 가중 합(Weighted Sum)하여 정보를 취합하는 원리를 배웁니다.
- **학습:** 가중치를 결정하는 정렬 모델(Alignment Model)도 역전파로 함께 학습됨을 파악합니다.

2 Essential Terminology

용어	의미	비유
Attention Weight	$\alpha^{(t,t')}$: t 번째 출력 시 t' 번째 입력의 중요도.	돋보기로 비추는 빛의 세기.
Context Vector	$c^{(t)}$: 인코더 상태들의 가중 평균.	번역 중인 단어의 연관 참고 자료.
Alignment Score	단어 간의 유사도/관련성 점수.	두 단어가 얼마나 '찰떡궁합'인가.

3 Core Concepts: 필요한 곳만 쳐다보기

3.1 1. The Intuition (직관)

기존 Seq2Seq는 인코더의 마지막 정보 하나만 디코더에게 던져줍니다. 반면, 어텐션 모델은 디코더가 단어를 생성할 때마다 인코더의 모든 은닉 상태($a^{(1)}, a^{(2)}, \dots$)를 다 참고합니다. 단, 중요한 것은 진하게(높은 가중치), 안 중요한 것은 흐리게 봅니다.

□ 번역가의 힐끔거리기 (직관적 비유)

- **기존:** 문장을 한 번 읽고 책을 덮은 뒤 암기해서 번역함. (병목 발생)
- **어텐션:** 번역문을 쓰면서 원문에서 지금 단어와 관련된 부분(예: 주어, 목적어)을 계속 힐끔힐끔 다시 보며 번역함.

3.2 2. The Architecture: Context Vector $c^{(t)}$

문맥 벡터 $c^{(t)}$ 는 인코더의 모든 은닉 상태 $a^{(t')}$ 의 가중 평균입니다.

□ Context Vector 공식 (수학적 원리)

$$c^{(t)} = \sum_{t'} \alpha^{(t,t')} a^{(t')}$$

- $\alpha^{(t,t')}$: 디코더 t 시점에 인코더 t' 시점을 보는 비중.
- $\sum_{t'} \alpha^{(t,t')} = 1$ (확률값).

4 Deep Dive: Computing Attention (수학적 원리)

그렇다면 가장 중요한 α (가중치)는 누가 정할까요? 사람이 정하는 게 아니라, 이조차 작은 신경망 (Alignment Model)이 학습합니다.

1. **Alignment Score** ($e^{(t,t')}$): 디코더의 이전 상태 $s^{(t-1)}$ 와 인코더 상태 $a^{(t')}$ 가 얼마나 어울리는지 점수를 매깁니다.

$$e^{(t,t')} = \text{dense_layer}([s^{(t-1)}, a^{(t')}])$$

2. **Softmax (Weights)**: 점수를 확률로 변환합니다.

$$\alpha^{(t,t')} = \frac{\exp(e^{(t,t')})}{\sum_{k=1}^{T_x} \exp(e^{(t,k)})}$$

5 Implementation: Bahdanau Attention

TensorFlow/Keras의 서브클래싱 방식으로 구현한 예시입니다.

```
1 import tensorflow as tf
2 from tensorflow.keras.layers import Layer, Dense
3
4 class BahdanauAttention(Layer):
5     def __init__(self, units):
6         super().__init__()
7         self.W1 = Dense(units) # Decoder state weight
8         self.W2 = Dense(units) # Encoder state weight
9         self.V = Dense(1)      # Score weight
10
11     def call(self, query, values):
12         # query: 디코더 은닉 상태 (Batch, hidden)
13         # values: 인코더 은닉 상태들 (Batch, Tx, hidden)
14
15         query_with_time = tf.expand_dims(query, 1)
16
17         # 1. 점수 계산 (Additive Attention)
18         score = self.V(tf.nn.tanh(self.W1(query_with_time) + self.W2(values)))
19
20         # 2. 가중치 계산 (Softmax)
21         attention_weights = tf.nn.softmax(score, axis=1)
22
23         # 3. 문맥 벡터 계산 (Weighted Sum)
24         context_vector = attention_weights * values
25         context_vector = tf.reduce_sum(context_vector, axis=1)
26
27         return context_vector, attention_weights
```

6 FAQ & Pitfalls

Q. 어텐션을 쓰면 모델이 무거워지지 않나요?

A. 네, 인코더의 모든 타임 스텝을 매번 연산해야 하므로 $T_x \times T_y$ 의 연산량이 추가됩니다. 하지만 성능 향상 폭이 워낙 커서 감수할 만한 비용입니다.

Q. 어텐션은 번역 말고 어디에 쓰이나요?

A. 이미지 캡셔닝에서도 쓰입니다. 단어를 생성할 때마다 이미지의 특정 구역을 쳐다보는 식이죠.

다음 단계 (Next Step)

어텐션은 RNN의 한계를 부수고 성능을 극대화했습니다. 하지만 사람들은 깨달았습니다. "어텐션이 이렇게 강력하다면, 굳이 느린 순차 방식(RNN)을 써야 할까? 그냥 어텐션만 쓰면 안 되나?"

여기서 딥러닝 역사를 바꾼 논문, "Attention Is All You Need"가 등장합니다. 다음 시간, RNN과 CNN을 모두 버리고 오직 어텐션만으로 무장한 현대 AI의 심장, [Transformer Network]에 대해 알아보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Problem:** Seq2Seq 인코더의 고정 크기 벡터 병목 현상.
2. **Solution:** 디코더가 매 순간 인코더의 모든 부분을 연관성에 따라 참고한다.
3. **Formula:** $c^{(t)} = \sum \alpha^{(t,t')} a^{(t')}$ (가중 평균).
4. **Benefit:** 긴 문장 번역 성능 비약적 상승 + 시각적 해석 가능.